

Natural Language BI Query Engine

Project Spec — for use with Cursor AI

Overview

A full-stack web application that allows a user to type a business question in plain English, automatically generates SQL using the Claude API, executes it against a SQLite database loaded with a retail sales dataset, and renders the results as an interactive chart. The app mirrors the architecture of enterprise Generative BI platforms like Vistra's AskBI.

Tech Stack

Frontend: React.js (Vite)

Backend: Python — FastAPI

Database: SQLite

AI: Claude API (claude-sonnet-4-20250514) via anthropic Python package

Charting: ECharts via echarts-for-react

Styling: Tailwind CSS

Dataset

Use any retail sales CSV with the following columns. Load it into a SQLite database called **retail.db** with a single table called **sales**.

- order_id
- order_date (TEXT, format YYYY-MM-DD)
- region
- product_category
- product_name
- quantity (INTEGER)
- unit_price (REAL)
- total_sales (REAL)
- customer_id

Application Flow

- User types a natural language question into a text input (e.g. "What were total sales by region last quarter?")
- Frontend sends the question to the FastAPI backend via POST /query

- Backend constructs a prompt and calls the Claude API
- Claude returns a SQL query as plain text
- Backend executes the SQL against retail.db using Python's sqlite3 module
- Backend returns the SQL query, raw results (as JSON), and a suggested chart type to the frontend
- Frontend renders results as an ECharts chart and shows raw data in a table below

Backend — FastAPI (main.py)

Endpoint: POST /query

Request body:

```
{ "question": "What were total sales by region?" }
```

Response body:

```
{
  "sql": "SELECT region, SUM(total_sales) FROM sales GROUP BY region",
  "results": [{"North", 42000}, {"South", 31000}],
  "columns": ["region", "total_sales"],
  "chart_type": "bar"
}
```

System Prompt

Build a system prompt that includes the full schema, today's date, and strict instructions. Example:

```
You are a SQL expert. Given a user's question, generate a single SQLite SELECT query.
```

```
The database has one table called `sales` with the following schema:
```

```
- order_id (TEXT)
- order_date (TEXT, format YYYY-MM-DD)
- region (TEXT)
- product_category (TEXT)
- product_name (TEXT)
- quantity (INTEGER)
- unit_price (REAL)
- total_sales (REAL)
- customer_id (TEXT)
```

```
Today's date is {today}.
```

```
Rules:
```

- Return ONLY the raw SQL query. No explanation. No markdown. No code fences.
- Only use SELECT statements. Never INSERT, UPDATE, DELETE, or DROP.
- Always use proper SQLite date functions for date filtering.

Chart Type Logic

- 1 numeric + 1 category column → bar

- 1 numeric + 1 date column → line
- 1 row, 1 numeric column → kpi (display number large)
- Multiple numeric columns → bar (grouped)
- Default → table only

Frontend — React (App.jsx)

UI Components

Query Input: Full-width text input + Ask button. On submit, POST to /query.

SQL Display: Read-only code block showing the generated SQL. Style with a dark terminal background.

Chart Panel: Renders an ECharts chart based on chart_type. Support bar, line, and kpi (large centered number).

Results Table: Raw rows in a clean HTML table with column headers below the chart.

Loading State: Spinner or skeleton while waiting for the API response.

Error State: Friendly error message if SQL fails or Claude returns something invalid.

ECharts Bar Chart Example

```
const option = {
  xAxis: { type: 'category', data: results.map(r => r[0]) },
  yAxis: { type: 'value' },
  series: [{ data: results.map(r => r[1]), type: 'bar' }]
};
```

Prompt Test Harness (test_harness.py)

A standalone Python script that validates query accuracy across 50+ test cases.

- Load question/expected-result pairs from test_cases.json
- For each question, call the /query endpoint
- Compare results to expected: check SQL contains key keywords and row count matches
- Print a pass/fail score and log any failures with generated SQL vs expected

test_cases.json Format

```
[
  {
    "question": "What is the total sales amount?",
    "expected_sql_contains": ["SUM", "total_sales"],
    "expected_row_count": 1
  },
  {
    "question": "Show sales by region",
    "expected_sql_contains": ["region", "GROUP BY"],
    "expected_row_count": 4
  }
]
```

```
}  
]
```

Project Structure

```
nl-bi-query/  
  ■■■ backend/  
    ■ ■■■ main.py          # FastAPI app  
    ■ ■■■ database.py      # SQLite connection + query execution  
    ■ ■■■ prompt.py        # System prompt builder  
    ■ ■■■ retail.db         # SQLite database (generated from CSV)  
  ■■■ frontend/  
    ■ ■■■ src/  
    ■ ■ ■■■ App.jsx  
    ■ ■ ■■■ components/  
    ■ ■ ■■■ QueryInput.jsx  
    ■ ■ ■■■ SqlDisplay.jsx  
    ■ ■ ■■■ ChartPanel.jsx  
    ■ ■ ■■■ ResultsTable.jsx  
    ■ ■■■ package.json  
  ■■■ test_harness.py  
  ■■■ test_cases.json  
  ■■■ load_data.py         # Script to load CSV into SQLite  
  ■■■ README.md
```

Setup Instructions (for README)

1. pip install fastapi uvicorn anthropic python-dotenv
2. Add ANTHROPIC_API_KEY=your_key to a .env file
3. Run python load_data.py to populate the SQLite DB from the CSV
4. uvicorn backend.main:app --reload to start the backend
5. cd frontend && npm install && npm run dev to start the frontend

Nice-to-Haves (if time allows)

- Dropdown of example questions to help users get started
- Query history sidebar showing past questions and their charts
- Retry button that re-prompts Claude if SQL returns an error
- Dark mode toggle

Build the backend first, then the frontend, then the test harness last.